MakersSoc Wireless

JJ Littleton

Wireless communication has many applications. This will show how to wire, configure and program transmitters and recievers for the nRF24L01 module. At the end there is a 2-way chat. While words or strings will be communicated, these is for demonstration as the signal can be used to communicate any data between devices. Such as button presses or values from a remote control.

Equipment needed:
Arduino x2
USB B cable x2
Arduino IDE
Male to Female Jumper Wire x14
nRF24L01 module x2

This workshop may be easier if participants work in pairs and take it in turn with who is programming a transmitter and who is programming the receiver.

Wireless Hello World

1. Connect the nRF24L01 component to the Arduino using the following pin layout:

nRF24L01	**Arduino**
VCC	3.3V
GND	GND
CE	Pin 9
CSN	Pin 10
SCK	Pin 13
MOSI	Pin 11
MISO	Pin 12
IRQ	Not Connected

[1] Pin out for nRF24L01

Make sure to connect the VCC pin to the 3.3V pin on the Arduino board and not the 5V pin. Connecting it to the 5V pin can damage the nRF24L01 module.

2. Install library

Open the Arduino IDE
Go to Sketch > Include Library > Manage Libraries
In the search bar, type "RF24"
Select "RF24" by TMRh20 and click on "Install"

3. Write the code

Plug one of the Arduino boards into the computer at the time. This is done for ease. Keep track of which board is transmitting and which is receiving, especially when uploading the code.

An alternative way would be to switch COM Ports by selecting each by Tools > Port.

This is the final code for the transmitter and receiver respectively.

It may be easier to pair up with someone and you take it in turns programming the transmitter and receiver.

Transmitter

```
#include <RF24.h>

// Create an instance of the radio
RF24 radio(9, 10);

const byte pipe[6] = "maker"; //Can be any 5
letter string
//CHANGE OTHERWISE EACH OTHER'S WILL INTERACT
WITH EACH OTHER

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openWritingPipe(pipe);
}

void loop() {
  const char *msg = "Hello World";
  radio.write(msg, strlen(msg));
  Serial.println("Message Sent");
  delay(1000);
}
```

Receiver

```
#include <RF24.h>

// Create an instance of the radio
RF24 radio(9, 10);

const byte pipe[6] = "maker"; //Can be any 5
letter string
//CHANGE OTHERWISE EACH OTHER'S WILL INTERACT
WITH EACH OTHER

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openReadingPipe(1, pipe);
  radio.startListening();
}

void loop() {
  if (radio.available()) {
    char msg[32] = "";
    radio.read(msg, sizeof(msg));
    Serial.println(msg);
  }
}
```

For both code bases:

The **`#include <RF24.h>`** line includes the RF24 library, which provides functions for communicating with the nRF24L01 module.

The line **`RF24 radio(9, 10);`** creates an instance of the RF24 class and Initialises it with pins 9 and 10 on the Arduino board. These pins are used to communicate with the nRF24L01 module.

The line **`const byte pipe[6] = "maker";`** sets the address of the pipe used for wireless communication. The pipe address is a unique identifier used by the nRF24L01 module to differentiate between different communication channels. In this case, the pipe address is a string of 5 characters, "maker", that can be changed to any other five character string as long as both the transmitter and receiver use the same address. **Please change this address so you do not interfere with others' boards. The pair of boards (transmitter and receiver) need to have the same address.**

For the transmitter:

The **setup()** function Initialises the serial communication at a baud rate of 9600, starts the radio communication using **radio.begin()** and sets the writing pipe address to the value of the **`pipe`** variable.

The **`loop()`** function runs repeatedly and sends a message "Hello World" wirelessly using the **`radio.write()`** function. The **`strlen(msg)`** function calculates the length of the message, which is required as an argument for the **`write()`** function. The function then prints "Message Sent" to the serial monitor using **`Serial.println()`**. Finally, the function waits for 1 second using **`delay(1000)`** before repeating the loop.

Overall, this code initialises the wireless communication using the nRF24L01 module and Arduino board and repeatedly sends a message "Hello World" wirelessly to another device.

Whereas for the receiver:

The **`setup()`** function initialises the serial communication at a baud rate of 9600, starts the radio communication using **`radio.begin()`**, sets the reading pipe address to the value of the **`pipe`** variable and starts listening for incoming messages using **`radio.startListening()`**.

The **`loop()`** function runs repeatedly and checks if there is any available message using the **`radio.available()`** function. If there is a message available, the function reads the message into a character array **`msg`** using the **`radio.read()`** function. The **`sizeof(msg)`** function determines the maximum length of the message that can be read into the **`msg`** array. Finally, the function prints the received message to the serial monitor using **`Serial.println(msg)`**.

Overall, this code initialises the wireless communication using the nRF24L01 module and Arduino board and repeatedly listens for incoming messages. When a message is received, the message is printed to the serial monitor.

4. View the messages being sent

Upload transmitter code and receiver code to two separate boards.

The message will be viewable in the serial monitor of the receiving board.
To do this, select the COM Port of the receiving board.
Then go to Tools > Serial Monitor.

If both boards are connected to the same computer, only one monitor will be visible at a time. Which monitor is dependent on the COM Port.

When both boards are powered, the receiver should the message repeatedly from the transmitter and this should be visible within the serial monitor. If the transmitting board's monitor is open, then "Message Sent" will be visible. Change the COM Port and the message should be visible.

Change the message in the code and see that the same is received.

Sending custom messages

1. Create two new Arduino files
   Similarly to before, **it may be easier to pair up with someone and you take it in turns programming the transmitter and receiver.**

   Be sure to name files appropriately so you can keep track. One will be the receiver code while the other will be the transmitter code.

2. Set up - Transmitter

   Write the following set up code

```
#include <RF24.h>

// Create an instance of the radio
RF24 radio(9, 10);

const byte address[6] = "maker"; //Can be any 5 letter string
//CHANGE OTHERWISE EACH OTHER'S WILL INTERACT WITH EACH OTHER

bool printOut = false;

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openWritingPipe(address);

  while (!Serial) {
    ; // wait for serial port to connect
  }

}
```

   This sets up everything that is needed to transmit.

3. Transmitting

   Write the following code under the set up function.

```
void loop() {
  if (printOut) { Serial.println("Awaiting message input"); };

  //Reads input from serial monitor and will repeat sending until another input message
is given
  if (Serial.available()) {
    String inputString = Serial.readStringUntil('\n');
    const char* inputChar = inputString.c_str();

    radio.write(inputChar, strlen(inputChar));
  }
```

```
  if (printOut) { Serial.println("Message Sent"); };
  delay(1000);
}
```

This code transmits a message.

You can upload this to one of the Arduino boards. This board will be the transmitting board.

4. Set up - Receiver

In a new file, write the following set up code for the receiver. Be sure to name files appropriately so you can keep track.

```
#include <RF24.h>

// Create an instance of the radio
RF24 radio(9, 10);

const byte address[6] = "maker"; //Can be any 5 letter string
//CHANGE OTHERWISE EACH OTHER'S WILL INTERACT WITH EACH OTHER

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openReadingPipe(1, address);
  radio.startListening();
}
```

This sets up everything that is necessary to receive messages.

5. Receiving

Write the following code the set up function.

```
void loop() {
  if (radio.available()) {
    char msg[32] = "";
    radio.read(msg, sizeof(msg));
    Serial.println(msg);
  }
}
```

This code receives, interrupts and displays the (received) message.

6. Upload the code

Your final code should look like this:

Transmitter

```
#include <RF24.h>

// Create an instance of the radio
RF24 radio(9, 10);

const byte address[6] = "maker"; //Can be any 5 letter string
//CHANGE OTHERWISE EACH OTHER'S WILL INTERACT WITH EACH OTHER

bool printOut = false;

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openWritingPipe(address);

  while (!Serial) {
    ; // wait for serial port to connect
  }

}

void loop() {
  if (printOut) { Serial.println("Awaiting message input"); };

  //Reads input from serial monitor and will repeat sending until another input message
is given
  if (Serial.available()) {
    String inputString = Serial.readStringUntil('\n');
    const char* inputChar = inputString.c_str();

    radio.write(inputChar, strlen(inputChar));
  }

  if (printOut) { Serial.println("Message Sent"); };
  delay(1000);
}
```

(Receiver code is on the next page)

Receiver

```
#include <RF24.h>

// Create an instance of the radio
RF24 radio(9, 10);

const byte address[6] = "maker"; //Can be any 5 letter string
//CHANGE OTHERWISE EACH OTHER'S WILL INTERACT WITH EACH OTHER

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openReadingPipe(1, address);
  radio.startListening();
}

void loop() {
  if (radio.available()) {
    char msg[32] = "";
    radio.read(msg, sizeof(msg));
    Serial.println(msg);
  }
}
```

Bidirectional Wireless communication: 2-way chat

1. Create a new file

2. Set up

   Write the following code.

```
#include <RF24.h>

// Create an instance of the radio
RF24 radio(9, 10);

const byte addresses[][6] = {"maker", "output"}; //Can be any 5 letter string
//const byte addresses[][6] = {"output", "maker"}; //Can be any 5 letter string
//CHANGE OTHERWISE EACH OTHER'S WILL INTERACT WITH EACH OTHER

bool printOut = false;
String const user = "JJ";
//String const user = "NotJJ";

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openWritingPipe(addresses[0]); //First address. Outgoing
  radio.openReadingPipe(1, addresses[1]); //Second address. Incoming

  while (!Serial) {
    ; // wait for serial port to connect
  }

  radio.startListening(); // Start listening for incoming messages
}
```

This is a general breakdown of additions to the code that were covered earlier in the labsheet.

#include <RF24.h>: This line includes the RF24 library.

`const byte addresses[][6] = {"maker", "output"};`: This line defines an array of two-byte arrays that represent the addresses of the two devices that will communicate. In this case, **`addresses[0]`** is the address of the transmitter device and **`addresses[1]`** is the address of the receiver device.

`bool printOut = false;`: Initialises a boolean variable **`printOut`** with the value **`false`**. This variable is not used in this code snippet and can be ignored. A Boolean is a something that can only be true or false.

`String const user = "JJ";`: Initialises a constant string variable **user** with the value "JJ". This variable is used later in the code to identify the sender of a message.

`void setup()`: The setup function is a standard function in Arduino code that is executed only once at the beginning of the program. In this code, the following is done:

`radio.openWritingPipe(addresses[0]);`: Opens a writing pipe for the radio module with the address of the transmitter device (**addresses[0]**).

radio.openReadingPipe(1, addresses[1]);: Opens a reading pipe for the radio module with the address of the receiver device (**addresses[1]**).

while (!Serial) {;}: Waits for the serial port to connect before continuing with the program.

Note that this code only sets up the communication between the devices and does not actually send or receive any messages. That is done in the **loop()** function, which is not shown in this code snippet.

3. Listening

Write the following code below the set up function

```
void listening() { //Receive
  if (radio.available()) {
    char msg[32] = "";
    radio.read(msg, sizeof(msg));
    Serial.println(msg);
  }
}
```

This code defines a function called **`listening()`** which is responsible for receiving messages through the RF24 radio module. The function first checks if there are any incoming messages available using the **`radio.available()`** function. If there is an incoming message, it creates a character array called **`msg`** with a length of 32 characters. The **`radio.read()`** function then reads the incoming message and stores it in the **`msg`** array, with the size of the message limited to the size of the array. Finally, the function prints the received message to the Serial Monitor using the **`Serial.println()`** function.

4. Sending

Write this below the listening function

```
void sending() { //Send

  //Reads input from serial monitor and sends message
  if (Serial.available()) {
    String inputString = Serial.readStringUntil('\n');
    String outgoingMessage = "[" + user + "] " + inputString;
    //const char* inputChar = inputString.c_str();
    const char* outgoingChar = outgoingMessage.c_str();

    radio.stopListening(); // Stop listening for incoming messages while transmitting
    radio.write(outgoingChar, strlen(outgoingChar));
    radio.startListening(); // Start listening for incoming messages again
```

```
    Serial.println(outgoingChar);
  }

}
```

The sending function does the following:

- The function starts by checking if there is any input available from the serial monitor using **`Serial.available()`**.
- If there is input available, it reads the input until it reaches a newline character (**`\n`**) using **`Serial.readStringUntil('\n')`**. This means the user has to send the message with a newline character at the end.
- It creates a string called **`outgoingMessage`** that contains the message with the user's name (defined earlier in the code) wrapped in square brackets.
- The string **`outgoingMessage`** is converted to a C-style string (a null-terminated array of characters) using **`c_str()`** and stored in a variable called **`outgoingChar`**.
- The function stops listening for incoming messages using **`radio.stopListening()`** to avoid collisions with the outgoing message.
- The outgoing message is sent using **`radio.write()`** with the address of the recipient and the length of the message as arguments.
- After the message is sent, the function starts listening for incoming messages again using **`radio.startListening()`**.
- Finally, the outgoing message is printed to the serial monitor using **`Serial.println(outgoingChar)`**.

5. Finalising

Finally, write this at the bottom of the code.

```
void loop() {

  sending();
  listening();
}
```

Upload the code

This code contains the main loop of the program which runs continuously. It calls two functions **`sending()`** and **`listening()`** in order to send and receive messages.
`sending()` function reads the input from the serial monitor until it detects a new line character. It then concatenates the user's name and the input string to form the outgoing message. The message is then transmitted using the **`radio.write()`** function with the outgoing message as an argument. Before sending the message, the radio stops listening for incoming messages and after the message is sent, it starts listening again.
`listening()` function continuously checks if there is any incoming message available by calling **`radio.available()`** function. If it detects an incoming message, it reads the message using **`radio.read()`** function and prints the message to the serial monitor using **`Serial.println()`** function.

The **`loop()`** function calls **`sending()`** and **`listening()`** functions continuously in an infinite loop, allowing for the user to send and receive messages in real-time.

Make sure that you have the addresses set correctly. If you want to transmit to a board, then the other board's receiving/reading address needs to have the same address.
For example, if two people wanted to communicate, one would have "**`addresses[][6] = {"maker", "twotwo"}`**" and the other person would have "**`addresses[][6] = {"twotwo", "maker"}`**".

Your final code should look like this:

```
#include <RF24.h>

// Create an instance of the radio
RF24 radio(9, 10);

const byte addresses[][6] = {"maker", "output"}; //Can be any 5 letter string
//const byte addresses[][6] = {"output", "maker"}; //Can be any 5 letter string
//CHANGE OTHERWISE EACH OTHER'S WILL INTERACT WITH EACH OTHER

bool printOut = false;
String const user = "JJ";
//String const user = "NotJJ";

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.openWritingPipe(addresses[0]); //First address. Outgoing
  radio.openReadingPipe(1, addresses[1]); //Second address. Incoming

  while (!Serial) {
    ; // wait for serial port to connect
  }

  radio.startListening(); // Start listening for incoming messages
}


void listening() { //Receive
  if (radio.available()) {
    char msg[32] = "";
    radio.read(msg, sizeof(msg));
    Serial.println(msg);
  }
}

void sending() { //Send

  //Reads input from serial monitor and sends message
  if (Serial.available()) {
    String inputString = Serial.readStringUntil('\n');
    String outgoingMessage = "[" + user + "] " + inputString;
    //const char* inputChar = inputString.c_str();
    const char* outgoingChar = outgoingMessage.c_str();

    radio.stopListening(); // Stop listening for incoming messages while transmitting
    radio.write(outgoingChar, strlen(outgoingChar));
    radio.startListening(); // Start listening for incoming messages again
    Serial.println(outgoingChar);
  }

}

void loop() {

  sending();
  listening();
}
```

[1] Dejan (no date) *NRF24L01 Pinout, nRF24L01 – How It Works, Arduino Interface, Circuits, Codes*. How To Mechatronics. Available at: https://howtomechatronics.com/tutorials/arduino/arduino-wireless-communication-nrf24l01-tutorial/ (Accessed: February 28, 2023).